ASSIGNMENT 3

Q10 - Nested Hoisting and Closures

JavaScript solution

```
// q10.js
"use strict";

function outer() {
  console.log(count); // undefined: outer's var is hoisted
  var count = 5;

  function inner() {
    console.log(count); // undefined: inner has its own hoisted var count
    var count = 10;
  }
  inner();
}
outer();

// Arrow-function version:
function outerArrowDemo() {
  console.log(count); // undefined
  var count = 5;

  const inner = () => {
    console.log(count); // 5: no local count is declared in the arrow
  };
  inner();
}
outerArrowDemo();

// Debug call stack:
// outer/outerArrowDemo is entered first; inner is pushed when called;
// after inner returns, execution resumes in outer; then outer returns.
```

Explanation / Debug notes

Original output is undefined, then undefined. Each function has its own execution context. The var count inside inner is hoisted within inner and shadows outer's count, so its first log is undefined. In the arrow version, no local count is declared, so the arrow reads the outer count (5) through lexical scope. Set breakpoints on each console.log and inspect Local/Closure variables and the Call Stack.

Run the code in the browser console or Node.js. Save each solution in its own .js file as required.